Agentic AI System Design Report

AI Level Design Triage Agent: A Human-in-the-Loop Agentic Workflow for 2D Platformer Level Candidates

Student: Robert Mayfield
Project: Udacity AI Masters Capstone, Agentic AI Applications

---

Overview

The AI Level Design Triage Agent is an advisory agentic AI workflow for evaluating a single 2D platformer level candidate against a designer's natural language brief. Given a brief and one candidate level, the system interprets the designer's intent, gathers factual evidence with deterministic analysis tools, reasons about the design tradeoffs, and recommends the next design action. The agent does **not** generate levels, edit levels, or claim that a level is objectively fun. Its job is to support a human designer's next decision.

The use case is level design triage: deciding whether a candidate should be accepted for playtest, revised, clarified, flagged as derivative, rejected for structural reasons, or escalated to human review. This framing is intentionally narrower than an autonomous level designer. The system is designed for an academic agentic workflow where reasoning, tool use, memory, safeguards, and observable behavior are the primary goals.

The governing design principle is:

> Deterministic code owns facts and hard safety. The language model agents own judgment, decision, and explanation.

That separation keeps the system auditable. Tools measure level properties such as validity, heuristic difficulty, novelty, pacing, safe zone presence, and local difficulty spikes. Agents interpret messy human intent, decide which facts matter in context, weigh tradeoffs, and produce the design recommendation. The output is advisory and human in the loop: the designer chooses whether to apply the recommendation.

Use Case and Why an Agentic Approach Is Appropriate

A platformer designer's goal is often expressed in natural language rather than as a fixed numeric target. For example, a designer may ask for "a beginner friendly first level that still feels a little exciting" or "a familiar level in my existing style, but not a direct copy." These are judgment heavy requests. The same measured fact can be interpreted differently depending on the brief: low novelty may be undesirable when the designer wants originality, but acceptable when the designer explicitly asks for style consistency.

This makes the task appropriate for an agentic workflow rather than a simple deterministic script. A script can compute facts, but it cannot reliably decide what those facts mean under a human design brief. The agentic layer is needed for four reasons:

1. **Intent interpretation:** The system must translate ambiguous or conflicting natural language design goals into a structured intent profile.
2. **Contextual tradeoff reasoning:** The system must decide whether facts such as difficulty, novelty, or pacing support or conflict with the brief.
3. **Tool-grounded judgment:** The system should call tools for facts instead of guessing level properties from text.
4. **Escalation and control:** The system must know when to ask for clarification or human review rather than inventing a confident answer.

This design is consistent with tool using agent patterns where language models interleave reasoning with actions and tool observations, as in ReAct style workflows (Yao et al., 2022), and with broader tool use

research showing why models should rely on external tools for tasks that require factual computation or specialized capabilities (Schick et al., 2023).

The game design side is also grounded in mixed initiative procedural content generation. PCGML research frames machine learning based procedural content generation as useful not only for autonomous generation, but also for critique, repair, content analysis, and mixed initiative design support (Summerville et al., 2018). In this project, the agent is not a replacement designer. It is a design direction assistant that helps a human decide what to do next.

Scope and Boundaries

The system handles one provided candidate level at a time. A candidate may be hand authored, generated by another system, or constructed as a scenario fixture. Project 6 does not run the level generator, integrate playtester feedback, rank multiple candidates, or perform multi round product level design iteration.

The Project 6 boundary is:

In scope for Project 6	Out of scope for Project 6
Interpret one natural language design brief	Generate new levels
Analyze one candidate level	Rank multiple generated candidates
Use deterministic tools for level facts	Integrate the Project 3 feedback classifier
Recommend a next design action	Run a full playtest feedback loop
Produce a revision prescription	Automatically edit the level
Log tool calls, reasoning trace, and final decision	Build a production application

This boundary keeps the agent's role honest. It evaluates and advises; it does not claim to solve level design automatically.

System Architecture and Agent Design

The architecture is a three agent, bounded evaluator-optimizer workflow over one triage pass. The system uses Pydantic AI because the task benefits from typed input/output models, structured tool calls, dependency injection, and schema-validated responses (Pydantic, n.d.). The implementation is model agnostic through a configurable `TRIAGE_MODEL` value. The selected model for the final tuned run is `gpt-4.1`.

The full-resolution diagram is also available as `architecture_diagram.png` in the project root.

The workflow is:

1. The **Intent Interpreter** converts the brief into a structured design intent profile.
2. The **Triage Director** calls deterministic tools, reasons over the facts and intent, and proposes a recommendation.
3. The **Critic** independently evaluates the recommendation and either approves it, requests revision, or escalates.
4. A thin deterministic **safety floor** enforces hard safety rules.
5. The system returns a `TriageSession` containing the final recommendation, tool facts, critic feedback, transcript, and audit log data.

The design deliberately separates agentic judgment from deterministic analysis. The deterministic layer never chooses the normal path action for a valid candidate. It only reports facts and enforces hard safety. The agents perform the judgment: interpreting intent, deciding which facts matter, resolving or escalating ambiguity, and explaining the recommendation.

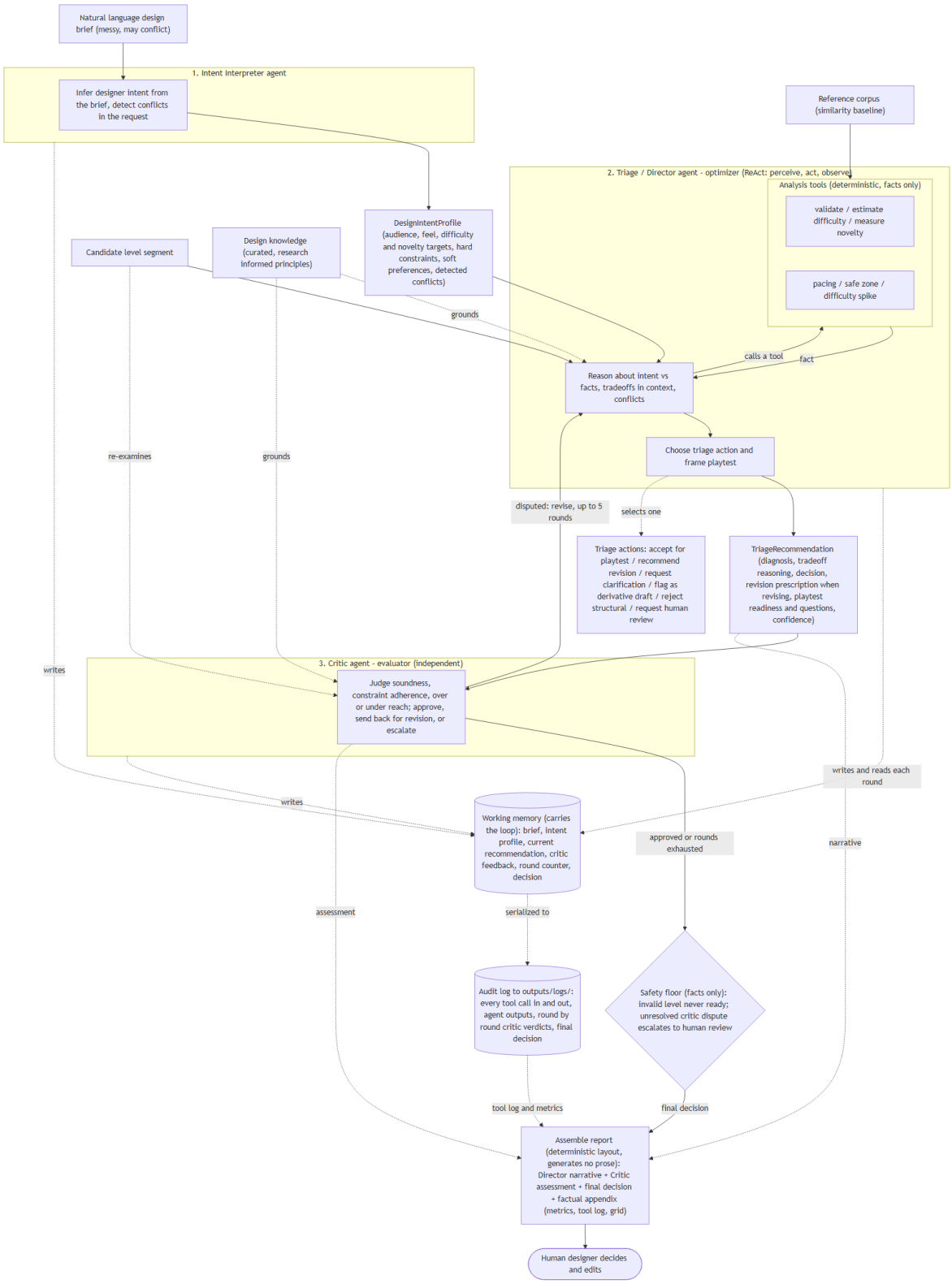

Figure 1: Architecture of the AI Level Design Triage Agent

Agent Personas

Intent Interpreter. The Intent Interpreter reads the designer brief and returns a structured `DesignIntentProfile`. The profile includes target audience, desired feel, difficulty target, novelty preference, hard constraints, soft preferences, and detected conflicts. Its most important role is not merely extracting keywords, but refusing to over infer. When a brief is contradictory or too vague, the Interpreter records that uncertainty instead of fabricating a confident target.

Triage Director. The Triage Director is the main ReAct style tool using agent. It calls analysis tools, observes the results, and reasons about the relationship between the candidate and the interpreted intent. It selects one terminal action and writes the design recommendation, including the diagnosis, tradeoff reasoning, revision prescription when needed, and playtest framing.

Critic. The Critic independently reviews the Director's recommendation against the interpreted intent, the measured facts, and the candidate level. It approves the recommendation, returns feedback to the Director for another bounded round, or escalates to human review. The Critic prevents the Director from becoming a one shot narrator and creates the evaluator-optimizer structure of the workflow.

Design Knowledge Grounding

A key design choice in this project was to avoid relying only on the language model's general training data for level design judgment. The deterministic tools measure facts about the candidate level, but those facts still need to be interpreted in a design context. For example, low novelty may be acceptable when the brief asks for style consistency, but it becomes a concern when the brief asks for a fresh or original segment. Similarly, flat pacing may be acceptable for an easy beginner segment, while a medium or hard segment may benefit from a stronger intensity ramp.

To support this interpretation layer, the system includes src/design_knowledge.py, which provides a curated DESIGN_GUIDANCE block injected into the Triage Director and Critic prompts. This guidance summarizes the design principles the agents should apply when reasoning over tool outputs: pacing and rhythm should be evaluated relative to the target audience, beginner levels should include an opening safe zone, local difficulty spikes can matter even when global difficulty is on target, layered hazards can reduce leniency, coins or other affordances can guide the player, and high similarity to known levels should be treated as a derivative-content risk rather than a pure validity failure.

This design keeps the agent grounded without turning this project into a retrieval augmented system. The system does not ask the model to invent level design principles from scratch. Instead, the report provides the formal citations, while the implementation uses a concise, project specific guidance block written in operational language that the agents can apply during triage.

Reasoning Loop

The reasoning loop combines two patterns. First, the Triage Director uses a reason-act-observe process: it reasons about what it needs to know, calls a tool, observes the tool result, and then continues its recommendation. Second, the Director and Critic form a bounded evaluator-optimizer loop. The Director proposes a recommendation, the Critic evaluates it, and the Director may revise the recommendation in response to Critic feedback. The loop is capped at five rounds to prevent open ended debate.

Memory and State Handling

Memory is implemented through the `TriageSession` Pydantic model. The session stores the original brief, candidate level identifier, interpreted intent, deterministic facts, current recommendation, critic feedback, number of rounds, final action, readiness status, tool-call log, and transcript. This is load bearing state, not just a record after the fact. If the Critic requests revision, the Director receives the prior recommendation and critic feedback on the next round.

The system also keeps a full audit trail. The transcript is normalized from Pydantic AI message history and records agent prompts, agent reasoning text, tool calls, tool results, deterministic workflow events, critic verdicts, and the final decision. A JSON audit log and readable transcript are saved for scenario runs. This improves transparency and supports human oversight, which is central to human AI interaction guidance (Amershi et al., 2019).

Tools

The Director has access to six deterministic tools. These tools return typed facts and do not decide the final recommendation.

Tool	Purpose	Why it matters
validate_candidate_level	Checks structure, dimensions, vocabulary, and pipe integrity	Invalid levels should not be marked ready for playtest
estimate_candidate_difficulty	Estimates difficulty using structural features	Provides a consistent difficulty signal
measure_candidate_novelty	Compares candidate against a reference library	Flags highly derivative candidates
analyze_candidate_pacing	Measures intensity across opening/middle/final thirds	Supports pacing-aware design reasoning
detect_candidate_safe_zone	Checks whether the opening gives beginners time to orient	Supports beginner-friendly triage
detect_candidate_spike	Detects local challenge layering such as gap plus enemy	Flags frustration risks that a global difficulty score may hide

The names above are the Python wrapper function names used in tool call logs and the tool registry. The Triage Director calls these tools by their registered Pydantic AI names: `check_validity`, `check_difficulty`, `check_novelty`, `check_pacing`, `check_safe_zone`, and `check_spike`. Both refer to the same underlying deterministic functions.

The level design tools are lightweight heuristics, but they are not arbitrary. Platformer analysis research emphasizes genre specific level structure, rhythm, pacing, and localized challenge composition (Smith et al., 2008). Level design pattern research also identifies concepts such as safe zones, guidance, layering, and pace breaking as common 2D design patterns (Khalifa et al., 2019). These concepts are encoded in the deterministic analysis tools and summarized in the design guidance injected into the Director and Critic prompts.

Decision Logic and Behavior

The system's decision logic is intentionally split between agentic judgment and deterministic guardrails.

The **agents** decide:

- how to interpret the designer's brief,
- whether the brief is clear enough to act on,
- which measured facts matter for the stated intent,
- whether the level should be accepted, revised, clarified, flagged, rejected, or escalated,
- how to explain the recommendation to the designer,
- what revision prescription or playtest question should be provided.

The **deterministic layer** decides only hard safety facts:

- a structurally invalid candidate cannot be ready for playtest,
- an invalid level is rejected for structural reasons,
- unresolved Critic disputes escalate to human review,

- final action/readiness values must stay within the allowed schema.

The six terminal actions are:

Action	Meaning
accept_for_playtest	Candidate is ready for human playtest with appropriate caveats
recommend_revision	Candidate has a fixable design issue; human should revise before playtest
request_clarification	Brief is too vague or contradictory for responsible action
flag_as_derivative_draft	Candidate may be useful as a draft but is too similar to reference material for final use
reject_structural	Candidate has a structural defect and should not be used as is
request_human_review	Signals are unresolved or outside the agent's reliable scope

The system uses revision prescriptions rather than automatic edits. When the Director recommends revision, it provides ordered suggested edits with reasons and a targeted playtest question. This keeps the system useful without taking creative control away from the designer. Mixed initiative content creation research supports this kind of human-software collaboration, where computational systems assist but the human remains the creative decision maker (Liapis et al., 2016; Smith et al., 2010).

Safeguards for Reliability, Transparency, and Control

The main reliability safeguard is the safety floor. It does not replace the agent's judgment, but it prevents unsafe outcomes. The safety floor enforces that a structurally invalid candidate is never accepted or marked ready for playtest. It also forces a not_ready status whenever the action is request_clarification, since revision cannot be prescribed before the designer's intent is known. It escalates unresolved critic disputes to human review and validates that final actions and readiness statuses are schema consistent.

The second safeguard is the independent Critic. The Director does not simply produce a recommendation and stop. The Critic checks whether the recommendation is grounded in the facts, respects the intent, and avoids overreach. If the Critic is not satisfied, the Director receives the feedback and must revise within the bounded loop. This supports reliability without creating an unbounded multi agent debate.

The third safeguard is transparency. Each scenario run produces a session record with the interpreted intent, measured facts, tool call log, agent recommendation, critic assessment, transcript, and final action. The report generator does not invent new prose; it assembles the agent authored diagnosis and critic assessment with a factual appendix so the reported metrics remain grounded in deterministic outputs.

The fourth safeguard is human creative control. The system never edits the level. It provides revision prescriptions and playtest questions, but the designer applies or rejects the advice. This reduces the risk of automation overreach and supports appropriate reliance, a key concern in human-AI systems (Amershi et al., 2019). The report also discloses that difficulty is heuristic and structural, not player validated, and that readiness for playtest is not proof that a level is fun. Documenting intended use and limitations follows the spirit of model/system reporting practices such as model cards (Mitchell et al., 2019).

Observed Behavior and Representative Scenario Walkthrough

The system was executed across seven representative scenarios. Each scenario pairs a natural language brief with a candidate level and an acceptable expected behavior. The scenarios were designed to cover normal cases, ambiguity, conflict, structural defects, low novelty, and local frustration.

One representative run is Scenario S3, the conflicting brief case.

Step	Observed behavior
Designer brief	"I want this to be easy and welcoming for brand new players, but also pack in lots of enemies and big gaps so it feels intense and hardcore."
Intent interpretation	Beginner audience and easy difficulty detected; conflict flagged between accessibility and the request for many enemies, large gaps, and an intense/hardcore feel.
Tool evidence	Structurally valid; medium difficulty; high novelty; safe opening; localized spike at a gap-plus-enemy challenge.
Director reasoning	The brief conflict blocked confident action; the same spike could be acceptable or unacceptable depending on the designer's priority.
Final recommendation	request_clarification — the Director refused to guess a priority.
Critic assessment	Approved; the brief contained a real unresolved conflict.
Readiness	not_ready — the design goal needed clarification before playtesting or revision.

This scenario demonstrates the system's core agentic behavior. A deterministic tool could report difficulty or spike information, but it could not determine that the brief itself was the blocking issue. The agentic decision was to refuse overconfident action and ask the designer to clarify the priority.

Scenario S1 demonstrates a different behavior. In that case, the candidate was valid and globally easy, but the first enemy appeared too early for a beginner friendly first level. The agent recommended revision rather than rejection or regeneration: move the first enemy later and preserve the main layout. This shows that the system can recommend a targeted design adjustment even when the global difficulty label appears acceptable.

Evaluation Results

Evaluation was scenario based and invariant based. Exact action matching is useful for visibility, but the task is a judgment task rather than a pure classification problem. Therefore, the final evaluation combines expected action comparison, invariant checks, transcript review, and saved scenario reports.

Scenario Suite Results

The tuned system reached the acceptable action on all seven scenarios, each in one round.

Scenario	Expected behavior	Actual action	Readiness	Match
S1 Beginner excitement tradeoff	Recommend revision	recommend_revision	revise_before_playtest	Yes
S2 Style consistency vs originality	Flag derivative draft	flag_as_derivative_draft	ready_for_playtest	Yes
S3 Conflicting brief	Request clarification	request_clarification	not_ready	Yes

Scenario	Expected behavior	Actual action	Readiness	Match
S4 Playtest readiness	Accept for playtest	accept_for_playtest	ready_for_playtest	Yes
S5 Structural defect	Reject structural defect	reject_structural	not_ready	Yes
S6 Preserve layout, reduce frustration	Recommend revision	recommend_revision	revise_before_playtest	Yes
S7 Ambiguous brief	Request clarification	request_clarification	not_ready	Yes

The scenario results show that the tuned agent handles several different decision types: accepting a clean candidate, prescribing revision, asking for clarification, rejecting a structural defect, and flagging a derivative draft.

Model Selection

Before tuning, three models were evaluated on identical baseline prompts. Each model ran seven scenarios three times, for 21 runs per model and 63 total passes across the comparison.

Model	Acceptable action match	Average rounds	Cost for model's 21 runs
gpt-4o-mini	0.43	3.7	$0.085
gpt-4.1	0.57	1.0	$0.304
gpt-5.1	0.52	1.0	$0.363

Match rates and average rounds are rounded to two decimal places; full precision values are in `outputs/eval_results/model_comparison.csv`.

`gpt-4.1` was selected because it had the highest baseline match rate, converged in a single round on average, and cost less than `gpt-5.1` in the comparison. `gpt-4o-mini` was not selected even though it was cheaper, because it tended to escalate too many cases to human review and failed to converge within the loop. That behavior can produce deceptively acceptable scores in escalation scenarios while still being poor design triage. The 3.7 average rounds figure also understates the escalation tendency: several gpt-4o-mini runs hit the per-run request budget and terminated in one round without converging, which pulls the average down.

Prompt Tuning

The model comparison exposed two major weaknesses. First, the Intent Interpreter sometimes resolved conflicting or vague briefs into confident targets instead of surfacing ambiguity. Second, the Director could act on candidate facts before respecting the detected conflict in the brief. Prompt tuning addressed these issues by making conflict and vagueness detection dominant in the Interpreter and making the Director check `detected_conflicts` before normal candidate triage.

After tuning, the selected model matched all seven acceptable action targets. This validation uses the same seven scenarios that identified the tuning targets, confirming that tuning resolved the intended behaviors rather than measuring generalization to new cases. This tuning did not change the deterministic safety floor. The improvement came from better agent judgment: detecting ambiguity, respecting that ambiguity, and refusing to over act when the brief was under specified.

Invariant Evaluation and Tests

A separate invariant evaluation suite checks properties that should always hold regardless of wording variation. These include valid action/readiness values, structural invalidity never being ready for playtest, revision

recommendations including a prescription, and final recommendations containing the required fields. The invariant suite passed all assertions, and the expected action match score was 1.000 for the tuned scenario run.

The project also includes a pytest suite for the deterministic modules, schemas, prompts, workflow, reports, safety checks, scenarios, and eval logic. The stored test run reports 87 passing tests with 1 warning in 4.39 seconds. The warning is a framework-level deprecation from pydantic-ai's async utilities and does not affect test results or execution.

Strengths and Failure Cases

The system's main strength is that it treats level triage as a contextual design judgment instead of a metric lookup. It can accept a candidate when the facts and intent align, recommend targeted revision when a design issue conflicts with the brief, request clarification when the brief is ambiguous or contradictory, and reject structurally invalid content. It also keeps a transparent transcript and factual appendix so the recommendation can be inspected.

A second strength is the explicit boundary between tools and agents. The tools keep the factual measurements consistent, while the agents decide what the measurements mean. This helps avoid both ungrounded LLM reasoning and overly rigid, rule based behavior.

One observed failure case came from the design knowledge layer itself. In an early run, the agent over applied the general idea that level intensity should ramp upward and recommended revision for a valid beginner segment that was already on target. This was not a tool failure: the deterministic facts were correct. The issue was that the injected design guidance was too general. I refined the guidance to make pacing target relative: low and fairly flat intensity is acceptable for easy or beginner segments, while stronger pacing ramps are more relevant for medium or hard targets. This improved the agent's behavior because it corrected the interpretation layer rather than changing the underlying metrics or safety rules.

The project also revealed several failure cases and limitations during development:

- **Cheap-model over-escalation:** `gpt-4o-mini` often escalated to human review, which made it appear partially correct on scenarios where escalation was acceptable, but it was not useful as a triage assistant.
- **Untuned intent overconfidence:** baseline prompts sometimes turned vague or conflicting briefs into confident structured targets. This was corrected through prompt tuning.
- **Heuristic blind spots:** the preserve layout/frustration case initially used a level where the designer's concern was not captured by any deterministic fact. The S6 fixture was redesigned to contain a deterministically detectable difficulty spike so the scenario cleanly exercises the spike-detection pathway under a preserve-layout constraint. The issue still shows a real limitation: heuristic tools can miss human experienced frustration that is not captured by structural measurements.
- **Model variability:** even with temperature zero, provider hosted LLMs are not guaranteed to be perfectly reproducible. This is why the evaluation emphasizes invariants and scenario properties rather than exact wording.

Ethical and Responsible Use Considerations

The main ethical concern is creative overreliance. A design assistant that produces confident recommendations could nudge a designer to defer creative judgment to the system. This is especially important in game design, where subjective experience, cultural context, theme, and player emotion matter. The system reduces this risk by staying advisory, refusing to edit the level automatically, documenting its limitations, and preserving human approval for every change.

A second concern is derivative content. Similarity and novelty tools can flag candidates that closely resemble reference material, but they do not provide legal or artistic certainty. The system therefore treats high similarity as a warning for human review, not as a final intellectual property judgment.

A third concern is false confidence. The difficulty score is heuristic and structural. It is not based on live player behavior or formal playtesting. The agent therefore frames readiness as readiness for **human playtest**, not as proof of quality or fun. This is consistent with responsible human-AI interaction principles that emphasize transparency, user control, and calibrated reliance (Amershi et al., 2019).

Limitations

The project is intentionally small scale. It evaluates one candidate level at a time and does not attempt to generate, rank, or edit levels. The reference library for novelty is limited, so novelty results should be treated as a warning signal rather than a comprehensive originality measure. The difficulty tool uses structural heuristics and cannot measure timing precision, player skill, aesthetics, theme, or actual completion probability. The safe zone and difficulty spike tools capture only simple design patterns and may miss more complex pacing or frustration problems.

The agent also depends on prompt quality and model behavior. Prompt tuning improved the selected model, but a different model or provider version could behave differently. The system mitigates this with typed outputs, invariant checks, prompt templates, and audit logs, but it does not eliminate model uncertainty.

Future Improvements and Integration

Near term improvements to the Project 6 agent include expanding the reference library, adding richer platformer metrics, improving detection of local frustration and guidance cues, and calibrating difficulty against real player data. The Critic could also be strengthened with more specific review rubrics for different target audiences, such as beginner, intermediate, and expert players.

Future systems can add an outer integrated product loop around this agent rather than redesigning the agent itself. The triage agent can become the design direction layer inside a larger system. That future system can add candidate generation, multi candidate ranking, playtester-feedback interpretation, and a full design iteration report. In that architecture, generation is one source of candidate drafts, feedback is another signal, and this Project 6 agent provides the judgment layer that recommends what the designer should do next.

Reproducibility and Professional Practice

The repository is organized so the system can be rerun and inspected. The implementation lives in `src/`, while `agentic_system.ipynb` demonstrates setup, tool behavior, model comparison, scenario execution, and final outputs. Prompt templates live in `prompt_templates/baseline/` and `prompt_templates/tuned/`, making the prompt-tuning surface visible and version controlled. Scenario fixtures live in `data/scenarios.json` and `data/candidate_levels/`. Outputs are saved under `outputs/`, including scenario reports, JSON logs, transcripts, model comparison results, evaluation results, and test reports.

The project includes a `requirements.txt` generated from the working environment. Scripts are provided for repeatable execution: `scripts/run_scenarios.py`, `scripts/run_evals.py`, `scripts/run_tests.py`, and `scripts/run_triage.py`. The notebook and scripts use the same underlying implementation, so the reviewer facing notebook is a demonstration layer rather than a separate version of the system.

References

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-AI interaction. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems.* https://doi.org/10.1145/3290605.3300233

Guzdial, M., Liao, N., & Riedl, M. (2018). *Co-creative level design via machine learning.* arXiv:1809.09420. https://arxiv.org/abs/1809.09420

Khalifa, A., de Mesentier Silva, F., & Togelius, J. (2019). Level design patterns in 2D games. *2019 IEEE Conference on Games (CoG)*, 1–8. https://doi.org/10.1109/CIG.2019.8847953

Liapis, A., Smith, G., & Shaker, N. (2016). Mixed-initiative content creation. In N. Shaker, J. Togelius, & M. J. Nelson (Eds.), *Procedural content generation in games.* Springer. https://doi.org/10.1007/978-3-319-42716-4_11

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency.* https://doi.org/10.1145/3287560.3287596

Pydantic. (n.d.). *Pydantic AI documentation.* Retrieved June 2026, from https://ai.pydantic.dev/

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). *Toolformer: Language models can teach themselves to use tools.* arXiv:2302.04761. Published at the Conference on Neural Information Processing Systems (NeurIPS), 2023. https://arxiv.org/abs/2302.04761

Smith, G., Cha, M., & Whitehead, J. (2008). A framework for analysis of 2D platformer levels. *Proceedings of the 2008 ACM SIGGRAPH Symposium on Video Games*, 75–80. https://doi.org/10.1145/1401843.1401858

Smith, G., Whitehead, J., & Mateas, M. (2010). Tanagra: A mixed-initiative level design tool. *Proceedings of the Fifth International Conference on the Foundations of Digital Games.* https://doi.org/10.1145/1822348.1822376

Summerville, A., Snodgrass, S., Guzdial, M., Holmgård, C., Hoover, A. K., Isaksen, A., Nealen, A., & Togelius, J. (2018). Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games, 10*(3), 257–270. https://doi.org/10.1109/TG.2018.2846639

Summerville, A., Snodgrass, S., Mateas, M., & Ontañón, S. (2016). The VGLC: The video game level corpus. *Proceedings of the 7th Workshop on Procedural Content Generation.* arXiv:1606.07487. https://arxiv.org/abs/1606.07487

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing reasoning and acting in language models.* arXiv:2210.03629. https://arxiv.org/abs/2210.03629